

构造，交互和通信

湘潭大学 谢东骏

July 16, 2026

概况

前言

构造

交互

CF679A

CF2129C

通信

结语

前言

前言

往年讲题的时候是没有这个板块的，因为这三类题目通常被直接划分为“思维题”。

很多学校/机构教算法的时候，考虑到这几类题目不像算法题那样，学了就能做题，加上也没有什么统一的解题方法论，就不讲了。

这就导致很多只学算法/只练算法题的同学（无论是高中生还是大学生）往往打 Codeforces 的时候会坠机，打出远低于自己掌握算法水平的表现，因为 CF 前期题要的就是思维难度。

根据 CF 的出题限制，CF div2 的前三题 A~C 是**禁止**出现线段树/复杂 DP 等知识点的。

出题人在从 D 开始往后的题目，才可以靠解题所需算法本身的难度来提升题目难度；前三题不给上算法了，那自然就要开始上思维难度了！

前言 (续 2)

于是乎，贪心、构造、交互还有通信就成了香饽饽。这几类题目没有统一的解题定式，非常适合拷打脑子转不过弯来的同学。

所以贪心构造成了 B~C 的常客，而交互通常在 DE 处等着大家，至于通信.....CF 去年才实装了这个东西，而它在国内比赛也是于 2025 年 ICPC 沈阳第一次出现。目前还没有很简单的通信题于正式赛事中出现。

好，由于贪心还是有一套对应方法论的（虽然高阶贪心就没有公式解了），我们今天就不讲，主要讲一讲构造、交互和通信的题型，以及对应的切入方式。由于时间限制我们会集中讲构造。

构造

介绍

构造，即题目给定一个构造对象（序列，矩阵，树，图...），对象中部分信息缺失。

再给定若干限制（序列某些位置的和、矩阵某些位置的关系、树 & 图上某些位置的关系）。

然后要求你的算法补充缺失部分，使最终得到的构造对象满足全部限制。完形填空。

铜牌构造：没有算法，充分发扬人类智慧即可想到答案，同时也没有太高的证明难度

银牌构造：构造过程需要使用算法加速或保证最优性，或在猜测时需要一些数学直觉、证明时需要一些数学能力

金牌构造：构造思路特别难想/用于优化的算法特别难/结论反直觉特别难证明

出线构造：证出来拿图灵奖

技巧

多练。

技巧不是不能总结，但基本都需要通过做题来感受这些技巧，熟能生巧后才能在后续的实战中感受那种自然而然想到的感觉。

也可以多做一些形如“海盗分金”之类的考公题来培养思维活性，类似脑筋急转弯，多走出一些思维定式有利于做出此类题目。构造题整体来说属于好玩的题型，确实可以当成脑筋急转弯来做。

所以构造题的训练整体属于一种拟合训练。我们接下来会按难度来让大家大面积地过一轮一些 trick 较为经典的题目。

CF743C Vladik and fractions

题目大意

Vladik 和 Chloe 决定要比一比谁的数学更好。Vladik 声称，对于任意正整数 n ，他都能将分数 $\frac{2}{n}$ 表示为三个不同的正分数的和，且这三个分数的形式都是 $\frac{1}{k}$ 。

请帮助 Vladik 实现这一想法。也就是说，给定 n ，请找出三个不同的正整数 x, y, z ，使得 $\frac{2}{n} = \frac{1}{x} + \frac{1}{y} + \frac{1}{z}$ 。由于 Chloe 无法验证太大的数，所以需保证所有的数都不超过 10^9 。

如果不存在这样的答案，请输出 -1 。 n ($1 \leq n \leq 10^4$)。

Sol

- 考虑所求式退化的情况：

$$\frac{1}{x} + \frac{1}{y} = \frac{1}{n}$$

- 这是经典的埃及分数问题，其对于项数只有 2 的情形（也就是上面的式子）存在唯一解：

Sol (续 2)

$$\frac{1}{n+1} + \frac{1}{n(n+1)} = \frac{1}{n}$$

- 当然这个式子有我们更喜闻乐见的形式,

Sol (续 3)



$$\frac{1}{n(n+1)} = \frac{1}{n} - \frac{1}{n+1}$$

, 是我们小学二年级就学过的裂项相消。

- 我们惊喜地发现此时左右两边再加上一个 $\frac{1}{n}$, 就能得到三个互不相等的分母凑出 $\frac{2}{n}$ 。
- 此时三个分母分别为 $n+1, n, n(n+1)$, 当且仅当 $n=1$ 时构造失效。

Sol (续 4)

- 这好办，我们证明 $n = 1$ 无解就好。 $\frac{1}{x} + \frac{1}{y} + \frac{1}{z}$ 时分别令 $x = 1, y = 2, z = 3$ 才能取到最大值，这个最大值是 $\frac{11}{6} < \frac{2}{1}$ ，所以 $n = 1$ 时无解。
- 以上推导过程属于**简化法**的范畴。我们先去考虑一个简单的问题，尝试解答这个简单的问题之后，推广到原问题的解法。

CF476D Dreamoon and Sets

题目大意

Dreamoon 喜欢玩集合、整数和最大公因数 (gcd)。gcd 定义为同时整除 a 和 b 的最大的正整数。

现有 S 为恰好包含四个不同的正整数的集合。如果对于集合 S 中任意一对不同元素 s_i, s_j , 都有 $\gcd(s_i, s_j) = k$, 则称 S 为秩 k 的集合。

给定 k 和 n , Dreamoon 想要用 1 到 m 间的整数, 构造 n 个秩为 k 的集合, 并保证每个整数最多只出现在一个集合中 (可以有一些整数没被用到)。请计算最小的 m 使得存在这样的方案, 并输出其中一种方案。

$1 \leq n \leq 10000, 1 \leq k \leq 100$ 。

Sol

- 我们针对“构造某属性最小方案”的题型提供一种通用思路：**下界分析法**
- 第一步：先分析出答案“至少”需要的代价是多少，这个最低代价称为“下界”；
- 第二步：针对“下界”进行构造，尝试构造出能稳定使代价取到下界的方案。
- 当然，如果你发现无法构造出稳定取到理论下界的方案，说明这题大概率不属于构造题，你可能需要请 DP 大能出山。
- 进入本题。假设最优答案存在，首先把答案中所有四元组除以 k ，那么所有四元组的最大公约数为 1。

Sol (续 2)

- 考虑任意一个四元组都只能存在一个偶数，那么奇数起码三个，于是我们答案的下界为 $6(n-1) + 5 = 6n - 1$ 。具体是，对前 $n-1$ 个四元组，其均需要取出连续的 6 个数并从中提取 3 个奇数，然后再从中提取 1 个偶数；对最后一个四元组，只需要提取到最后一个奇数即可。
- 显然不存在更低的总代价了，然后我们一个不小心啊，发现**上面的证明本身就是一种构造**。
- 具体地，由于 $\gcd(a, b) = \gcd(a, b-a)$ ，我们贪心的让方案内的差尽可能小，于是我们直接把最近的三个奇数和一个偶数放在一个四元组内，即第 $i(0 \leq i < n)$ 组为 $6i+1, 6i+2, 6i+3, 6i+5$ 。

Sol (续 3)

- 由 $\gcd(a, b) = \gcd(a, b - a)$ 不难得知两两之间 $\gcd$ 都为 1，于是方案合法。
- 以上两题是 Math 一点的构造题，实际上我们还有一些更传统风格的构造。

B3753 [信息与未来 2019] 堆栈计算机

题目大意

有一种新型的堆栈计算机，计算机的内存是一个初始为空的数列，计算机支持三种操作：

- 1 - 将整数 1 放入数列的尾部。任意时刻都可执行此操作。
- dup - 将数列尾部的数字复制一份，放入数列尾部。只有数列非空时才能执行此操作。
- add - 取出（并删除）数列尾部的两个数字，相加后放回数列的尾部。只有数列中至少有两个数字时才能执行此操作。

给定一个正整数，你需要输出一个长度不超过 200 的程序（操作的序列），它在执行结束后，恰好得到一个长度为 1 的数列，并且数列中的数字恰好为 n 。
 $1 \leq n \leq 10^9$ 。

Sol

- 递归处理。定义函数 $\text{solve}(n)$ 为得到恰好一个 n 的函数。
- 当 $n = 1$ 时，直接加入 1。
- 否则，先 $\text{solve}(n/2)$ 一下，使得栈内有一个 $\lfloor \frac{n}{2} \rfloor$ ，然后 dup 一下 add 一下，栈内就有了一个 $2 \cdot \lfloor \frac{n}{2} \rfloor$ 。如果 n 是奇数需要加一个 1 随后再 add 一下，即可得到 n 。
- 因为最多递归调用 $O(\log n)$ 即 29 层，极端情况下需要输出约 $4 \times 29 = 116$ 行，题目的限制绰绰有余。
- 这道题目其实已经有了一些交互题的影子。

2025 辽宁省赛 L

题目大意

给定一个正整数 n 和两个长度为 n 的排列 r, c 。

请您求出一个满足下列所有条件的矩阵：

- 矩阵有 n 行 n 列，元素都是在 $[0, n]$ 内的整数。
- 第 i 行元素的 MEX 值是 r_i 。
- 第 i 列元素的 MEX 值是 c_i 。

可以证明一定存在这样的矩阵。 $1 \leq n \leq 2 \times 10^3$ 。

2025 辽宁省赛 L (续 2)

一组整数 $a_1, a_2, \dots, a_k$ 的 MEX (Minimum Excluded) 是该组中未出现的最小非负整数 x 。例如, $\text{MEX}([2, 2, 1]) = 0$ (因为 0 不在数组中), $\text{MEX}([3, 1, 0, 1]) = 2$ (因为 0 和 1 在数组中, 但是 2 不在), $\text{MEX}([0, 3, 1, 2]) = 4$ (因为 0、1、2 和 3 在数组中, 但是 4 不在)。

Sol

- 性质：对于一个合法的矩阵，交换两行以及对应的 r_i ，由于每行、每列的整数组不变，交换后得到的矩阵依然合法。同理，交换每列后得到的矩阵依然合法。
- 基于此，我们可以对矩阵进行一些行列交换使其变成一个规律的形式，例如对于从 1 到 n 所有的 i ，使得 $r_i = c_i = i$ 。下面以 $n = 5$ 为例。

Sol (续 2)

	1	2	3	4	5
1					
2					
3					
4					
5					

Sol (续 3)

- 只需对这种形式的矩阵找出一种合法的构造方式，即可通过一些行列交换为所有情况构造合法矩阵。
- 法一：正确性：第 i 行中有且仅有 $[0, i - 1]$ 中的所有整数，所以可以保证每行的 MEX 均为 i 。列同理。形式化：对于第 i 行，第 j 列的位置，填入 $\min(r[i], c[j]) - 1$

Sol (续 4)

	1	2	3	4	5
1	0	0	0	0	0
2	0	1	1	1	1
3	0	1	2	2	2
4	0	1	2	3	3
5	0	1	2	3	4

Sol (续 5)

- 法二：正确性：第 i 行有且仅有 $[0, n] - i$ 中的所有整数，所以可以保证每行的 MEX 均为 i 。列同理。形式化：对于第 i 行，第 j 列的位置，填入 $(r[i] + c[j]) \bmod (n + 1)$ 。

Sol (续 6)

	1	2	3	4	5
1	2	3	4	5	0
2	3	4	5	0	1
3	4	5	0	1	2
4	5	0	1	2	3
5	0	1	2	3	4

Sol (续 7)

- 本题为辽宁省赛第 6 题，通过即金。

来追梦周赛 T3

题目大意

01 有一个长度为 n 的排列，其中 n 为奇数。

这个排列现在缺少了一个数 x ，只剩下了 $n - 1$ 个数。

01 想把这个 $n - 1$ 个数平均分为两组，使得每一组的和在对 n 取模后相等。

$1 \leq N \leq 2 \times 10^6$

Sol

- 题目没有要求我们报告无解，难道说在 $n \geq 5$ 时一定有解？
- 先不急证明，既然给了我们这个突破口，就想想看怎么在知道一定有解的前提下构造答案。
- 假设存在解，我们随机选择任意 $\frac{n}{2}$ 个数放入序列 S_1 ，其余数放入序列 S_2 。
- 对于 S_1 中的任意一个数，必存在某个数值 x ，若 $x \in S_2$ ，则交换二者即可平衡总和。显然该 x 位于 $[1, n]$ 之内。
- 我们计算出该 x ，那么 x 需满足： $x \neq 0$ ， x 未被删除，且 $x \notin S_1$ 。

Sol (续 2)

- 我们的划分是随机的，因此对于任意的 x ，该 x 有约 $\frac{1}{2}$ 的概率恰好落在 S_2 中。

$$\left(\frac{1}{2}\right)^{60} \approx 10^{-18}$$

也就是说，检查序列中的前 60 个数，其对应的 x 全都不符合要求的概率仅为一亿亿分之一 (10^{-18})。

Sol (续 3)

- 因此我们不断随机化重划 S_1 和 S_2 , 然后检查对应的 x 是否存在即可。
- 关于 $n \geq 5$ 时一定有解的证明: 这里空间太小写不下。
- 以上解法用到了**随机化**的思想。**随机化思想**启示我们, 如果我们存在一种算法使得它仅以**极小**的概率出错, 那么我们就可以用这个算法狠狠过题。

ABC272G Yet Another mod M

题目大意

给出一个长度为 N 的序列 A ，其中 A 的每个元素均为正整数且互不相同。你需要选择一个的正整数 M ，满足 $3 \leq M \leq 10^9$ ，并执行一次下列操作：

- 对于 $1 \leq i \leq N$ ，将 A_i 替换为 $A_i \bmod M$ 。

若能找到一个数 M 使得序列 A 中存在一个数 x ，且对于 $1 \leq i \leq N$ ，满足 $A_i = x$ 的数量大于 $A_i \neq x$ 的数量，输出这个 M ，否则输出 -1 。

Sol

- 假如存在一个合法的 M ，那么原序列就可以表示为 p 个形如 $k \cdot M + x$ 的数和 $n - p$ 个不能被如此表示的数。其中 $2p > n$ ， x 为一个小于 M 的常数。
- 那么选取这 p 个数中的任意两个作差，得到的一定是 M 的倍数。
- 还可以证明，如果 M 是合法的，那么 M 的任意一个因数也是合法的。
- 那么我们有了一个简单的想法：枚举两个数 u, v ，相减得到一个初始的 M ；扫一遍原序列，将枚举到的数与 u 作差并与 M 计算 gcd，如果这个 gcd 大于 3 就接受，否则认为扫到的这个数不在 p 中，跳过。最后检查有多少个数（包括 u, v ）被接受了，如果满足条件就可以直接打印得到的 gcd 了。
- 时间复杂度 $O(n^3)$ ，还不能接受。

Sol (续 2)

- 注意到我们要求解的是主元素，也就是说这个合法集合的大小是大于原序列一半的。如果我们随机地选取两个数，那么这两个数都落在这个合法集合中的概率是 25%。
- 所以我们可以采取随机化的策略，每次随机两个数 u, v ，然后按照上面的方法扫原序列并检查答案。多次随机，如果有解就直接打印答案，反之认为无解即可。
- 随机的次数可以按照效率来选取。单次扫描原序列是 $O(n)$ 的，因此我们也可以把随机次数选取在 $O(n)$ （例如取到 5000）来使得总效率控制在 $O(n^2)$ 。而这么做的话误判（即实际有解，但由于运气过于不好导致随机的两个数从来没有同时落入合法集合中）的概率最坏是 $(\frac{3}{4})^{5000} \approx 5 \times 10^{-3011}$ ，完全可以接受。

Sol (续 3)

- 这题也是随机化思想的一大应用。

交互

交互

好，我们开始打交。

交互题是需要选手程序与测评程序交互来完成的任务的题目。

一类常见的情形是，选手程序向测评程序发出询问，并得到其反馈；测评程序可能对选手的询问作出限制，或调整应答策略来尽可能增加询问次数，这也给题目带来了更多变化。

ACM-ICPC 中常见的交互形式为 I/O 交互，即通过输入输出的方式进行询问和回答。

P1733 猜数 (IO 交互版)

题目大意

题面较长，需要访问链接 (<https://www.luogu.com.cn/problem/P1733>) 来阅读。

Sol& 要点

- 解法是显然的，直接二分即可。这样一来我们的询问次数不会超过 30 次，低于题目要求。
- 本题作为模板题，向我们展示的要点在于：
- 第一：注意观察询问次数的限制，这通常直接提示做法，就像传统题需要注意数据范围一样。
- 不同的是，交互题的提示力度更大，例如当 $N = 1000$ 而限制次数恰好略高于 $\lfloor \log_2 N \rfloor$ ，这就是明着来告诉你：你每次的询问需要像二分一样至少使得子问题规模减半，你接下来的思路就应该导向每次询问如何获取至少一半规模的信息。

Sol& 要点 (续 2)

- 第二：注意交互格式。交互题由于评测机制复杂，如果你的交互不符合格式，往往会得到意料之外的结果（但绝不会是 AC），比如莫名其妙 WA、TLE、RE 甚至 MLE。你不仅需要注意输入输出的格式，还要注意题目特别提示的**刷新缓冲区**。作为 C++ 选手，我们通常只需要保证**不关闭流同步并保证使用 `endl` 换行**，即可保证不会出现交互 I/O 问题。

CF679A

题目大意

Bear Limak 想了一个隐藏的数字——一个在区间 $[2, 100]$ 内的整数。你的任务是判断这个隐藏的数字是质数还是合数。

对于 $x > 1$ 的整数，如果 x 恰好有两个不同的约数（1 和 x 本身），那么它是质数。如果 $x > 1$ 不是质数，则称为合数。

你最多可以询问 20 次关于隐藏数字的约数的问题。每次询问时，你应输出区间 $[2, 100]$ 内的一个整数。如果你的整数是隐藏数字的约数，系统会回复 “yes”，否则回复 “no”。

例如，如果隐藏数字是 14，那么系统仅在你输出 2、7 或 14 时回复 “yes”。当你完成询问后，应输出 “prime” 或 “composite” 并结束你的程序。

Sol

- 由于每个质数都有且只有两个因数，所以直接枚举目标数的因数，并尝试统计个数。我们被限制最多询问 20 次，不能枚举所有数，但可以尝试只枚举质数，这样一来如果一个数有至少两个不同的质因子，那么它一定不是质数。
- 我们发现枚举的范围只到 $\frac{100}{2}$ 以内的质数即可，原因是目标数一旦有了超过这个范围的质因子，再乘任意一个其他因子就爆了上界，所以含有 $[50, 100]$ 内质因子的目标数一定是质数。
- 这样一来我们把范围内的质数列举出来，发现只有 15 个数。然而我们发现这样的判断方式不是充分的，因为形如 4, 9, 16... 这样的完全平方数就只有一个质因子，我们判不掉。

Sol (续 2)

- 解决方案是把 2, 3, 5, 7 的平方 4, 9, 25, 49 也放进询问集合中。这样一来, 这些平方数自身就会被查出来; 同时, 16 会被 2, 4 筛到两次, 81 会被 3, 9 筛到两次, 因此其他的平方数也被解决了。
- 至此我们的询问集合大小为 19, 满足题意。

CF843B

题目大意

给定一个按升序排序的单链表。你需要找到链表中大于等于 x 的最小整数。更正式地说，链表基于包含 n 个元素的数组构建。第 i 个元素包含两个整数： $value_i$ 表示该节点的整数值， $next_i$ 表示下一个节点的下标（如果当前节点为链表尾节点，则 $next_i = -1$ ）。链表是有序的，也就是说如果 $next_i \neq -1$ ，则 $value_{next_i} > value_i$ 。

你会得到链表中元素个数 n ，第一个元素的下标 $start$ ，和整数 x 。

你最多可以进行 2000 次如下两种类型的查询：

CF843B (续 2)

- ? i ($1 \leq i \leq n$) —— 查询第 i 个元素的 $value_i$ 和 $next_i$;
- ! ans —— 输出问题的答案: 链表中大于等于 x 的最小整数, 或者如果没有这样的整数, 输出! -1。此语句后你的程序应终止。

请编写一个程序解决这个问题。 $1 \leq n \leq 50000$

Sol

- 链表最多有 5×10^4 个元素，但我们只能询问 1999 次，并且只能获取元素的后一个元素，所以普通的遍历整个链表的方法不可用。直接设法逼近目标元素的位置只有一种方法：随机撒点。
- 对于 $n \leq 2000$ 的情况直接枚举；当 $n > 2000$ 时，我们直接撒 1000 个点，此时这些点之间的期望距离很小。我们可以从小于 x 的最大值开始向后遍历，可以证明在到达下一个点之前我们就已得到答案。遍历过程中一旦找到大于等于 x 的元素，就可以直接推出答案。
- 同时由于 Codeforces 具有 hack 机制，很多人会刻意卡掉没有初始化随机种子的代码，所以在 `random_shuffle()` 函数前需要 `srand((size_t)new char)`。

CF2129C

题目大意

有一个长度为 N 的括号序列 S ，你不知道它具体的形态，每次可以向交互库询问，选中其中不超过 1000 个可重复的位置，以任意顺序排列这些位置构成一个序列，提交给交互库。

然后交互库返回一个整数，表示你查询的序列对应构成的括号序列中有多少个连续子串是合法括号串，括号串的定义不再赘述。要求你在 K 次询问内获得 S 的所有信息，然后报告 S 。

保证 S 不全部由 (或) 中的某一种构成。

EZ: $K = 550$

MED: $K = 200$

HD: $K = 100$

Sol

- 交互绝世好题，几乎在一道题内完成了交互题思考路径的完整引导——前两个难度等级依次启发下一题的思路，因此我们接下来也按顺序介绍三个难度的解法。
- Solve #1
- 不妨扩大题目的限制，想想怎么在 1000 步左右解决问题，以及我们需要什么。
- 题目允许我们使用重复的下标，也允许我们任意排列这些下标，如果我们知道某个位置 p 具体是（还是）的话（这里假设是），那我们就可以对每个下标 i 询问？ $2 \leq i \leq p$ ，如果返回值是 1 那么 i 就是（，返回值是 0 那么 i 就是）。

Sol (续 2)

- 该如何快速找到一个这样的位置 p 呢? 以找 $)$ 为例, 我们发现固定查询左端点为 1, 查询序列 $1, 2, 3, \dots, len$ 得到的返回值关于 len 的取值是单调不降的。所以可以二分 len 来找到第一个有 $($ 与之匹配的 $)$, 从而也就找到了一个 $)$ 。一个特例是如果原序列内不存在一个子串为合法括号序列的话, 那么下标为 1 的位置一定是 $)$, 原因是题目保证了至少分别存在一个 $($ 和 $)$ 。
- 找到 p 这一步需要开头判一次刚刚说的特殊情况, 加上二分的 $\lceil \log_2(1000) \rceil = 10$ 次查询, 已经用去 11 次查询。不过这是简单版本, 查询次数给足了余量。

Sol (续 3)

- 接下来盯着题目给出的 550 次查询限制，提示我们试着在一次查询中确定两个位置的值。既然前面获得了 p ，我们先尝试构造形如 $?*)$ 或者 $?)*$ （问号和星号分别表示两个要查询的位置）的查询，但发现这样行不通，因为如果返回值是 1 的话我们无法区分是哪个位置没有匹配。那么我们尝试让返回值具有“差异化”，让我们能知道左右两边的匹配情况。一种合理的构造方案是 $?)**$ ，这样当结果为 4 时，说明两个位置都是 (，为 2 说明第一个位置是) 第二个位置是 (，为 1 反之，为 0 说明两个位置都是)。
- 至此第一个难度的问题解决。 $N = 1000$ 时使用不超过 511 次查询。
- solve #2

Sol (续 4)

- 在上面的做法中，我们是通过差异化给两个位置分配的右括号数量，来让返回值里包含具有两个位置区分度的信息。注意到单次查询的序列长度可以达到 1000，那么顺着这个思路，加入更多的查询位置，然后将其也和前面的位置“差异化”分配右括号就行。

Sol (续 5)

- 不难想到二进制拆分。形如?) 的查询得到的返回值是 $0/1$, ???) 得到的是 $0/2$, ?????) 得到的返回值是 $0/4$, ... 将这些查询序列“拼”起来就可以根据返回值的二进制表示得到哪些位置是 (还是)。然而我们不能简单地将它们直接连接起来, 原因是括号串 ()(()) 的返回值是 4 而不是 3, 首尾相连的合法串会对答案产生额外贡献。因此我们需要在每个 ???...?))...) 后再单独添加一个) 来保证这样的合法串无法首尾连接产生贡献。

Sol (续 6)

- 那么我们在长度 1000 的询问序列中可以同时获得几个位置的答案呢？很遗憾只有至多 8 个位置（而不是直觉得到的 10 个位置），原因是假设我们要同时获得 a 个位置的答案，那么由等比数列求和，按照上面的构造方式我们至少需要长度为 $2^{a+1} + a - 3$ 的询问序列，在 $a = 9$ 时已经超出限制。
- 但是一次查询获取 8 个位置的信息足以我们通过本题中等难度的版本。我们的总查询数量不超过 $\lceil \frac{1000}{8} \rceil + 11 = 136$ 次。
- solve #3
- 离正解只差最后一步了。可以感觉到二进制拆分的思路是对的，但是长度限制让上面的构造很难继续优化了。

Sol (续 7)

- 这个时候考虑二进制拆分为什么可以用一个非负整数映射集合。因为二进制下每个位置加权后的值严格大于所有低位权值之和，也就是 $2^k > 2^k - 1 = \sum_{i=0}^{k-1} 2^i$ ，不存在一个位置的值可以被低位值的某一种组合加出，从而保证每个位置信息独立。
- 也就是说，如果我们存在一种分配每位权值的方案，满足上面的条件，那么这样的分配也是可以用一个非负整数去映射集合的。
- 回到刚才的构造，如果我们能用更少的串长让一个位置的权值更大（也就是合法括号串的数量更大），且满足上面的条件，就可以在单次查询塞下更多位置。

Sol (续 8)

- 那么我们的目光移向形如 $()()() \dots ()$ 的串, 用长度为 $2 \cdot len$ 的串长即可获得 $\frac{len \cdot (len+1)}{2}$ 个合法括号串。然而给第一个位置的 len 设为 1, 给第二个位置的 len 设为 2, 第三个位置设 3,是行不通的, 容易出现这样得到的每个位置的权值不满足本文第二段提出的要求。
- 那不满足我们就让它满足好了。打表, 令 len_1 为 1, 要求 len_i 满足 $2 \cdot \frac{len_{i-1} \cdot (len_{i-1}+1)}{2} \leq \frac{len_i \cdot (len_i+1)}{2}$ 的前提下尽可能小, 然后我们看看每个位置最小的 len 是多少, 以及要用多少查询长度。
- 对前 12 位打表代码如下。

Sol (续 9)

```
void solve(){
    ll x=1,sum=1;
    cout<<x<<' ';
    for(ll i=2;i<=12;i++){
        ll y=x;
        while(y*(y+1)<x*(x+1)*2)y++;
        cout<<y<<' ';
        x=y;
        sum+=y;
    }
    cout<<endl;
    cout<<sum*2<<endl;
    return ;
}
```

Sol (续 10)

- 输出结果:

```
1 2 3 5 8 12 18 26 37 53 76 108
698
```

Sol (续 11)

- 那么也就是说，按照输出结果第一行的方式分配每一位的权值，只需要使用 698 个长度就可以用得到的结果表示 12 个不同的位置是（还是），同时我们还要额外的 11 个位置在每一位间塞入）来防止中等难度问题解法中提到的额外贡献问题。可以根据输出结果发现塞 13 个位置就不行了。
- 那么总查询数量不超过 $\lceil \frac{1000}{12} \rceil + 11 = 95$ 次，足以通过本题全部难度版本。
- 回顾本题，我们完成了“怎么用询问获取信息”——“怎么用询问获取尽可能多的信息”——“怎么充分利用性质使获取效率最大化”的完整优化流程。这套流程可以在日后绝大多数交互题中使用。

通信

通信

由于 XCPC/CF 赛场还没出现过较为简单的通信题，通信能讲的内容有限，我们本章就只讲解通信的题型和基本思路。

通信题可以理解为一种特殊的交互题，在通信题中，你需要实现两份不同的程序：发送端和接收端，通常是发送端对已知信息进行加密，然后接收端负责解密。

通常，你的程序会先执行一次发送端，将得到的加密信息交给交互库；交互库对加密信息进行一定处理，再交给接收端。若接收端能正确根据所得信息还原答案，则视为通信成功，顺利得分。

通信 (续 2)

在这个过程中，交互库扮演“蒙蔽者/攻击者”的角色：首先它隔离了发送端和接收端，使它们不能直接在同一次运行中的内存里作弊直接沟通；其次，它通常不会让发送端的信息如愿原样到达，会进行一定程度的干扰。因此发送端通常需要加入一些加密措施来使得信息具有抗干扰能力。

几个没有出处的口胡例题

题目大意

1. 给发送端一个长度为 N 的 01 串，你可以发送一个长度为 $N+1$ 的 01 串，但这个串有概率被交互库逐位翻转。如何让接收端能还原出初始串？

答：在原串后面加一个 0 作为校验位，若该位最后变成了 1，说明整个串被翻转了，翻回去即可。

2. 给发送端一个长度为 N 的 01 串，你可以发送一个长度为 $N+2$ 的 01 串，但这个串有概率被交互库按下标整体反转。如何让接收端能还原出初始串？

答：在开头放一个 1，结尾放一个 0，若最终串以 0 开头，说明这个串被反转了，转回去即可。

2025 沈阳站热身赛 A

题目大意

给你一个 $[1923, 2024]$ 之间的年份，要求你用一个长度为 8 的 01 串传递这个年份。这个 01 串有可能被按下标整体翻转。如何让接收端能还原出目标年份？

Sol

- 总共有 $2024 - 1923 + 1 = 102$ 种不同年份。我们直接暴力枚举所有 01 串，然后初始的当前年份设定为 1923，每次都让一个新的 01 串尝试映射到当前年份上，形成一组新的对应关系。如果当前串没被映射到，我们就让这个串和它的翻转形式一起被映射到当前年份，然后当前年份自增，直到我们对完所有的合法年份。
- 共有 $2^8 = 256$ 种不同的 01 串，最坏情况下每两个 01 串对应一个年份，一共能映射出 128 种不同年份，完全符合要求。实际上，有很多串是回文的，回文串可以独自映射一个年份，所以实际所需 01 串的数量只会更少。

【UER #8】打雪仗

题目大意

小 A 有一个长度为 $2n$ 的 01 字符串 S , 小 B 有 $\{1, \dots, 2n\}$ 这些下标中的 n 个: $p_1, \dots, p_n$ 。

小 A 和小 B 可以相互之间发送字符 (但只能发送 “0” 或者 “1” 两种)。
请为小 A 和小 B 设计一种通信方式, 使得小 B 最终能知道这 n 个下标所对应的字符 $S[p_1], S[p_2], \dots, S[p_n]$ 。

允许多轮通信, 但两人中发送字符总数较多的那个不应发送超过 m 个字符。
 $1 \leq n \leq 1000, m = 1350$ 。

Sol

- 将序列分为三块，小 B 用二进制告诉小 A 需要信息最多的那个块，小 A 直接把这一整块原样发过来。然后剩下两个块由小 B 告诉小 A 具体需要传输哪些位置，小 A 再传对应位置即可。
- 对于小 B：用二进制传递一个 $[0, 2]$ 之间的整数只需要 2 位；接着对于剩下的两块分别用 01 来表示这一位要还是不要，总计需要这两块的块长的位数也就是 $\frac{2 \cdot 2}{3}n$ 位，加起来不超过 1350。
- 对于小 A：把信息最多的一整块发过去需要 $\frac{2}{3}n$ 位，发送剩下的位置总数不超过 $\frac{2}{3}n$ 位，加起来也不超过 1350。

结语

结语

今天所讲的内容旨在让大家对这些题目类型有一定概念。思维类题目的 Trick 千千万万，但大范围地去多见一见题目、了解更多的技巧和思维模式还是能很快地覆盖主流题目出题方向的。希望大家能课下通过有目的有规划的训练来提升自己的水平。感谢聆听。